El Lenguaje



Polimorfismo

Polimorfismo

- *Polimorfismo es la capacidad de los diferentes objetos a reaccionar de una manera individual al mismo mensaje.*
- *Su utilización deriva de la que se hace en los lenguajes naturales.*
- *El mensaje cerrar es interpretado en forma diferente si se trata de una puerta, una cuenta bancaria o la ventana activa operando una computadora.*

Polimorfismo

- *C++ tiene tres mecanismos de polimorfismo estático (en tiempo de compilación)*
  - *Sobrecarga de operadores*
  - *Sobrecarga de funciones*
  - *Plantillas*
- *También existen otras formas que se relacionan con lo que se denomina ligadura dinámica.*

Polimorfismo

- *Típicamente el polimorfismo ocurre con clases que están relacionadas por la herencia.*
- *El verdadero polimorfismo es dinámico, lo que representa un mecanismo muy poderoso.*
- *La sobrecarga es una elección que se toma en tiempo de compilación, la liga-dura dinámica en tiempo de corrida.*

Ligadura dinámica

- *Las ligaduras dinámicas amplían el concepto de polimorfismo.*
- *En las ligaduras dinámicas el significado de un mensaje depende del objeto que lo recibe y esto puede ser determinado solamente en tiempo de corrida.*
- *Para esto se utilizan las funciones virtuales.*

Funciones virtuales

- *El concepto se asocia con punteros a objetos.*
- *Supongamos que tenemos una clase base y dos clases derivadas que tienen definidas una misma función show().*
- *¿Cuál será la salida del programa siguiente?*

```
#include <iostream>  // P05-01.cpp
using namespace std;
class Base {                                  // clase base
  public:
  void show() { cout << "Base" << endl; }  // función normal
};

class Derv1 : public Base {                   // clase derivada 1
  public:
    void show() { cout << "Deriv1" << endl; }
};

class Derv2 : public Base {                   // clase derivada 2
  public:
    void show() { cout << "Deriv2" << endl; }
};

int main( void ) {
  Derv1 dv1;           // objeto de clase derivada 1
  Derv2 dv2;           // objeto de clase derivada 2
  Base* ptr;           // puntero a la clase base

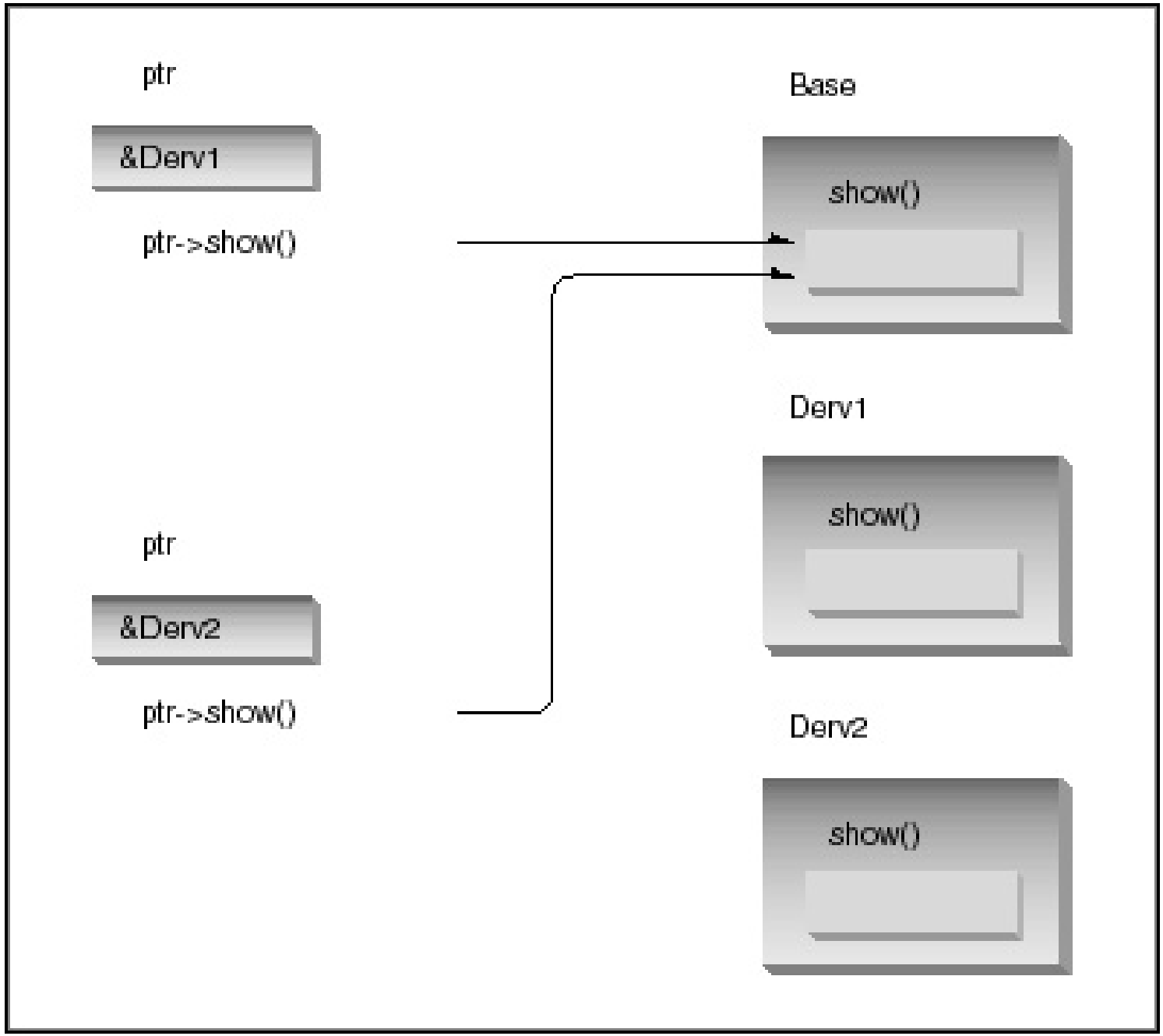
  ptr = &dv1;          // dirección de dv1 en el puntero
  ptr->show();         // ejecuta show()
  ptr = &dv2;          // dirección de dv2 en el puntero
  ptr->show();         // ejecuta show()
}
```

Funciones virtuales

- *La salida es:*

```
Base
Base
```

- *Esto no debería sorprender ya que la clase del puntero es: Base.*
- *Es lícito que pueda dirigir el puntero a un objeto de clase derivada, pero si se activa un método este corresponde de-cididamente a la clase Base.*


ptr
&Derv1
ptr->show()
Base
show()
Derv1
show()
ptr
&Derv2
ptr->show()
Derv2
show()

Funciones virtuales

- *Hagamos un pequeño cambio en el programa anterior.*
- *La salida ahora será:*

```
Deriv1
Deriv2
```

- *Han actuado los métodos de las clases derivadas, no el de la clase base.*
- *El método es elegido en base al contenido del puntero y no el tipo del puntero.*

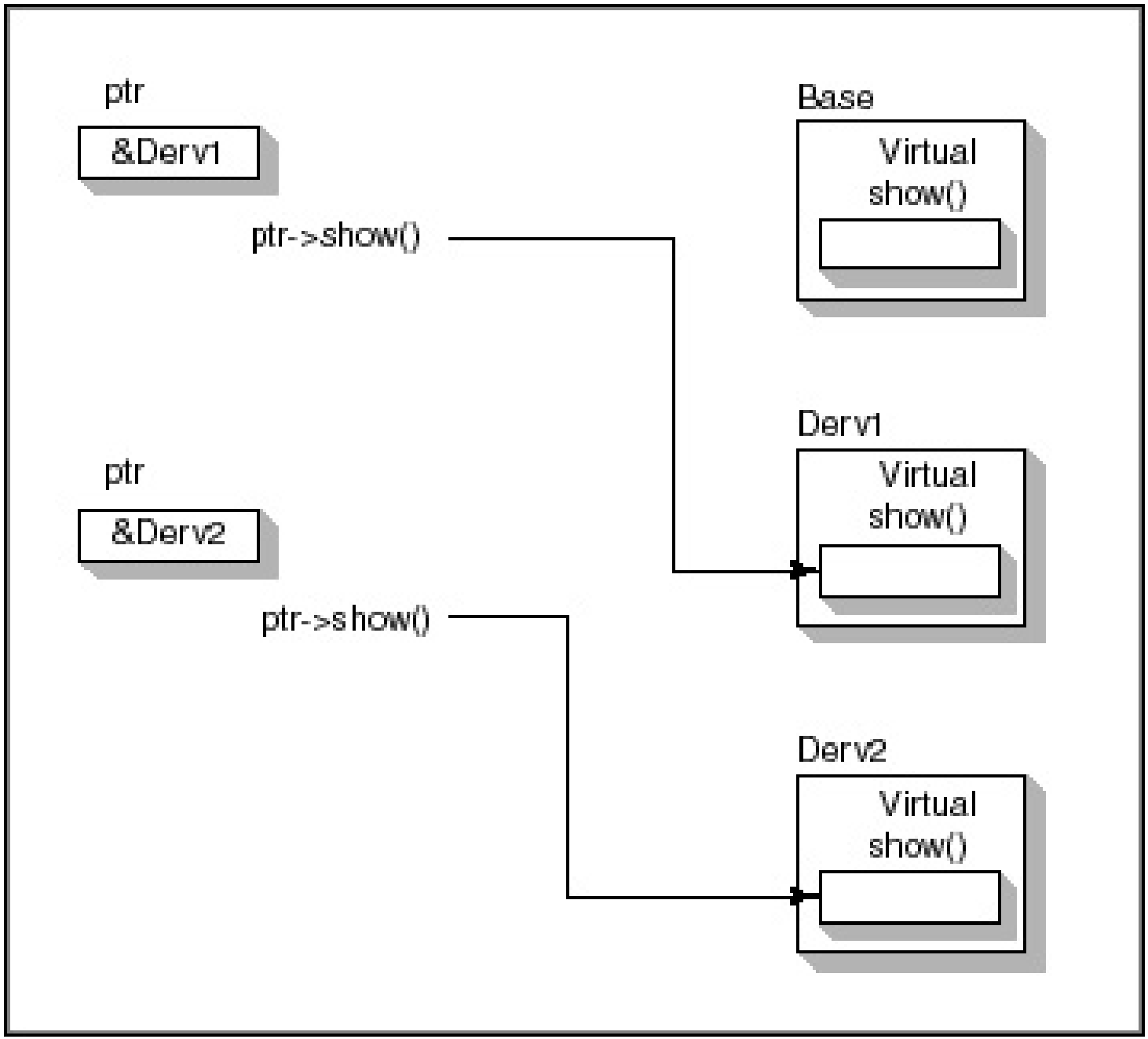
```cpp
#include <iostream>  // P05-02.cpp
using namespace std;
class Base {                                  // clase base
  public:
  virtual void show() { cout << "Base" << endl; } // función virtual
};

class Derv1 : public Base {                   // clase derivada 1
  public:
    void show() { cout << "Deriv1" << endl; }
};

class Derv2 : public Base {                   // clase derivada 2
  public:
    void show() { cout << "Deriv2" << endl; }
};

int main( void ) {
  Derv1 dv1;           // objeto de clase derivada 1
  Derv2 dv2;           // objeto de clase derivada 2
  Base* ptr;           // puntero a la clase base

  ptr = &dv1;          // dirección de dv1 en el puntero
  ptr->show();         // ejecuta show()
  ptr = &dv2;          // dirección de dv2 en el puntero
  ptr->show();         // ejecuta show()
}
```


ptr
&Derv1
ptr->show()
Base
Virtual
show()
Derv1
Virtual
show()
ptr
&Derv2
ptr->show()
Derv2
Virtual
show()

Ligadura tardía

- *Si se observa detenidamente, en el primer caso visto, no hay problemas para determinar el método a utilizar, porque este queda fijado en tiempo de compilación con la clase base.*
- *En el caso recién planteado, no se sabe cuál método se utilizará, ya que esto se decide de acuerdo a la clase de objeto que apunta el puntero ahora.*

Ligadura estática

- *En contrapartida la ligadura tradicional se denomina ligadura temprana o ligadura estática.*
- *La ligadura tardía lleva cierta sobrecarga, estimada en aproximadamente un 10%.*
- *Los beneficios que proporciona son muy superiores a la sobrecarga mencionada.*

Cómo funciona

- *Cuando se llama un método para este objeto, el sistema pasa la dirección del objeto que lo convocó a la función.*
- *Esta dirección está disponible a la función mediante el puntero "this". Esto es lo que usa la función para acceder los datos del objeto.*

Cómo funciona

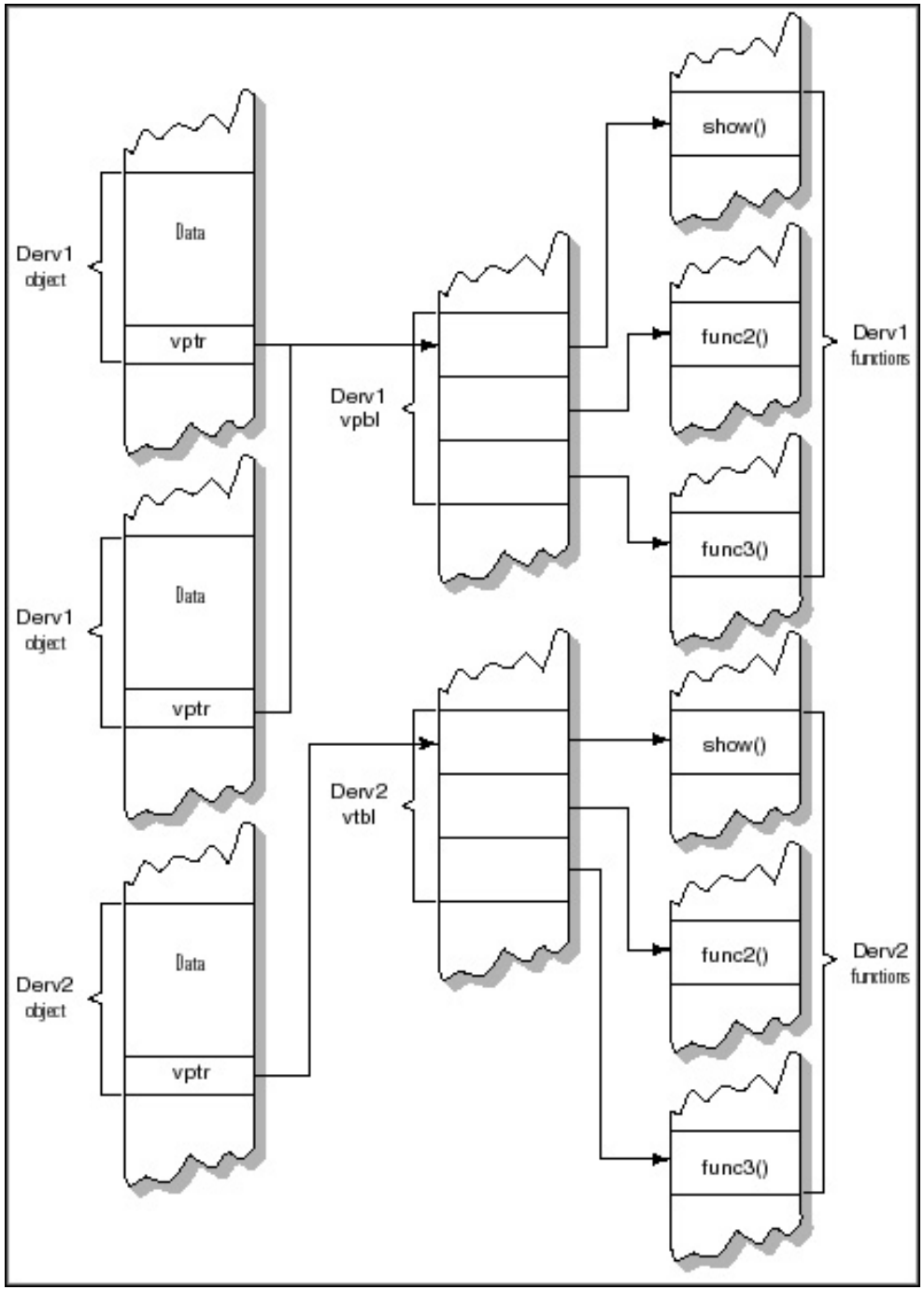
- *La dirección en "this" es generada por el sistema cada vez que se llama una función miembro, no está almacenada en el objeto y no ocupa espacio en memoria.*
- *El puntero "this" es la única conexión necesaria entre un objeto y sus funciones miembro normales.*

Cómo funciona

- *Con funciones virtuales las cosas son diferentes.*
- *Cuando se especifica una clase deriva-da con funciones virtuales el sistema crea una tabla llamada "tabla virtual (vtable)" con direcciones de función.*
- *En el ejemplo anterior, las clases Derv1 y Derv2 cada una tiene su propia tabla virtual.*

Cómo funciona

- *Hay una entrada en cada tabla virtual para cada función virtual en la clase. En este caso como ambas derivan de "Base", tienen las mismas funciones virtuales.*
- *No obstante las direcciones en cada tabla son diferentes para las dos clases.*


Derv1
object
Data
vptr
Derv1
vpbl
show()
func2()
func3()
Derv1
functions
Derv1
object
Data
vptr
Derv2
vtbl
show()
func2()
func3()
Derv2
functions
Derv2
object
Data
vptr

Puntero vptr

- *Cada objeto de Derv1 y Derv2 cuando son construidos, contienen un puntero extra, llamado "vptr" que se agrega a sus datos normales.*
- *Esto es así para los objetos que corresponden a una clase con funciones virtuales.*
- *Estos objetos ocupan un espacio ligeramente mayor que los comunes.*

Arreglos de punteros a objetos

- *La forma más común de usar funciones virtuales es con arreglos de punteros a objetos.*
- *En el ejemplo que sigue vemos la facilidad con que se llaman las diferentes funciones show() dentro del lazo for de iteración.*
- *Esto es polimorfismo en su mejor expresión.*

```
#include <iostream>  // P05-03.cpp
using namespace std;

class Base {                                  // clase base
  public:
  virtual void show() { cout << "Base" << endl; } // función virtual
};

class Derv1 : public Base {                   // clase derivada 1
  public:
    void show() { cout << "Deriv1" << endl; }
};

class Derv2 : public Base {                   // clase derivada 2
  public:
    void show() { cout << "Deriv2" << endl; }
};

int main( void ) {
  Base* arrBase[2];    // arreglo de punteros a clase Base
  Derv1 dv1;           // objeto de clase derivada 1
  Derv2 dv2;           // objeto de clase derivada 2

  arrBase[0] = &dv1;   // dirección de dv1 en el arreglo
  arrBase[1] = &dv2;   // dirección de dv2 en el arreglo

  for(int j=0; j<2; j++ ) {   // ejecuta show() para todos
    arrBase[j]->show();       // los elementos del arreglo
  }
}
```

Ligadura tardía

- *Decimos que la decisión es diferida hasta el tiempo de corrida.*
- *Estrictamente las funciones a llamar serán:*

```
show()
Derv1::show()
Derv2::show()
```

- *Ligadura dinámica y ligadura tardía son sinónimos.*

El puntero "this"

- *Todo objeto tiene un puntero que apunta a si mismo llamado "this".*
- *La definición es la siguiente:*

```
nombre_clase *this;
```

- *Existen oportunidades en que un método que corresponde a un objeto necesita acceder al mismo objeto.*

El puntero "this"

- *La utilización de this es implícita, pero a veces necesitamos de que sea explícita por lo que debemos usar la palabra this.*
- *Por ejemplo en:*

```
miobj.valor();
```

- *El puntero derreferenciado *this está dentro de la función, por esto no debemos aclarar el ámbito cuando usamos miembros del mismo objeto.*

Plantillas

Plantillas

- *Plantillas son el mecanismo subyacente para las clases contenedoras moder-nas.*
- *Las plantillas solucionan algo que hasta ahora hemos hecho con sobrecarga de funciones.*
- *Si bien la sobrecarga es un adelanto muy importante, también es cierto que se debe escribir muchas variantes.*

Plantillas de función

- *Por ejemplo para una función que calcula el valor absoluto:*

```
int abs( int n ) {          // valor absoluto de un entero
  return ( n<0 ) ? -n : n; // si n es negativo, retorna -n
}


long abs( long n ) {        // valor absoluto de un entero largo
  return ( n<0 ) ? -n : n; // si n es negativo, retorna -n
}


float abs( float n ) {      // valor absoluto de un float
  return ( n<0 ) ? -n : n; // si n es negativo, retorna -n
}
```

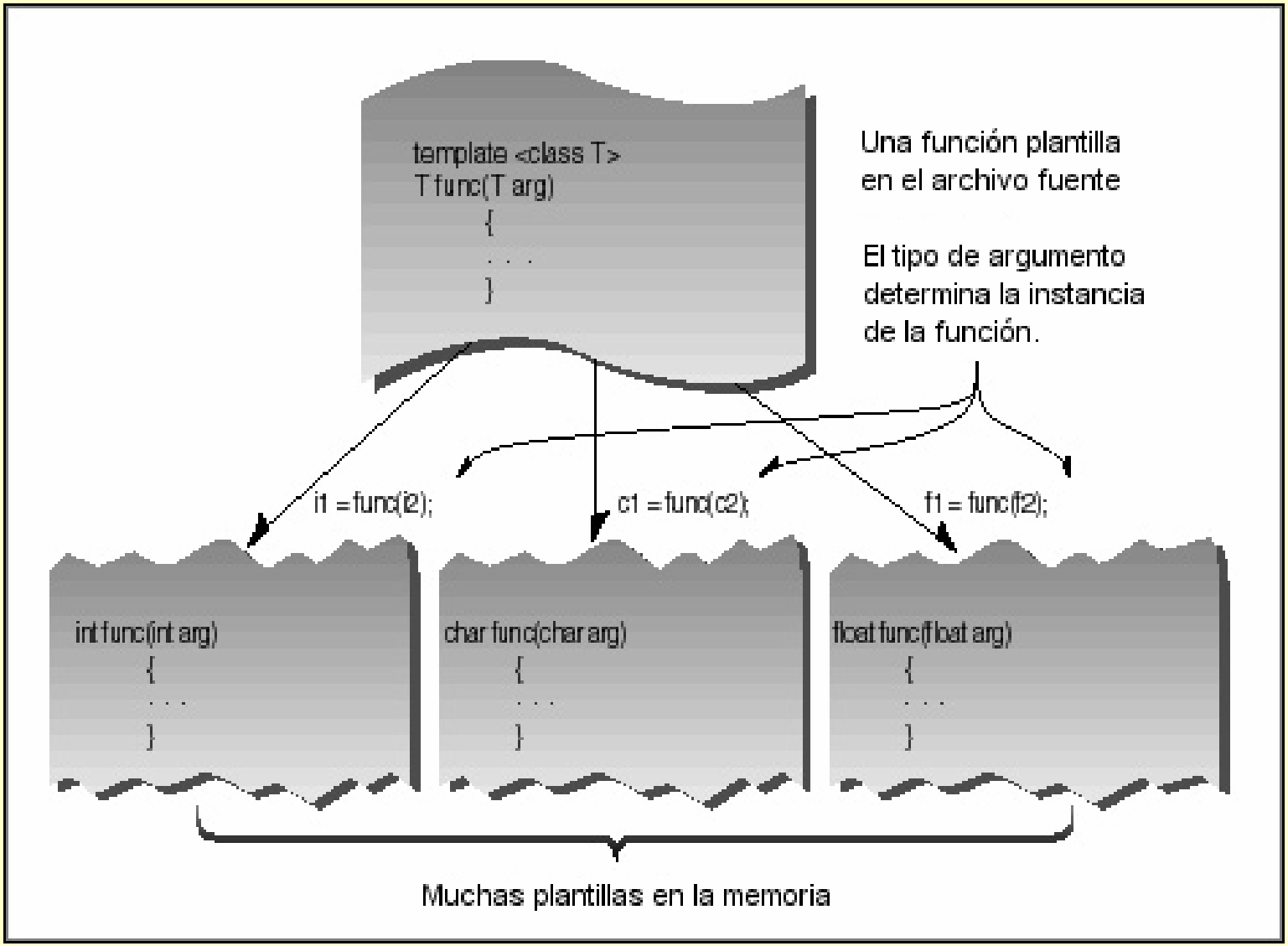
Plantillas de función

- *Como vemos, el cuerpo de la función es igual en todos los casos, pero debemos duplicarlas porque trabajan con datos de tipos diferentes.*
- *"C" ni siquiera permite la sobrecarga y en la biblioteca "math.h" tenemos:*

```
abs()       fabs()       fabsl()
labs()      cabs()       ....
```

- *A pesar de la ventaja de C++, resulta engorroso.*

Sería interesante tener algo como lo siguiente:



Plantillas de función

- *Esto quiere decir que escribimos una sola función plantilla (template) para el valor absoluto de todos los tipos que necesitemos utilizar.*
- *Esto no solo es cierto para los tipos básicos sino también para los que defina el usuario.*
- *Veamos el programa siguiente:*

```
#include <iostream>  // P05-04.cpp

using namespace std;

template <class T>              // plantilla de función
T abs(T n) {
  return (n < 0) ? -n : n;
}

int main( void ) {
  int int1 = 5;
  int int2 = -6;
  long lon1 = 70000L;
  long lon2 = -80000L;
  double dub1 = 9.95;
  double dub2 = -10.15;

  cout << "abs(" << int1 << ") = " << abs(int1);   // abs(int)
  cout << "\nabs(" << int2 << ") = " << abs(int2); // abs(int)
  cout << "\nabs(" << lon1 << ") = " << abs(lon1); // abs(long)
  cout << "\nabs(" << lon2 << ") = " << abs(lon2); // abs(long)
  cout << "\nabs(" << dub1 << ") = " << abs(dub1); // abs(double)
  cout << "\nabs(" << dub2 << ") = " << abs(dub2); // abs(double)
  cout << endl;
}
```

```
Salida:
abs(5) = 5
abs(-6) = 6
abs(70000) = 70000
abs(-80000) = 80000
abs( 9.95 ) = 9.95
abs(-10.15) = 10.15
```

Plantillas - Sintaxis

- *Para declarar una plantilla usamos la palabra reservada: template, y a continuación va la declaración de la función, denominada función plantilla.*
- *El tipo a ser reemplazado es lo que se indica con: "T", que significa: cualquier tipo.*
- *Por costumbre siempre se usa la "T", pero podría llamarse de cualquier forma*

Plantillas de función

- *Esto se ve en la siguiente declaración:*

```
template <class Coco>                    // plantilla de función
Coco abs( Coco n ) {
  return (n < 0) ? -n : n;
}
```

- *¿Qué hace el compilador con esto?*
- *En principio una declaración de plantilla no genera código, solamente lo "memo-riza" para uso futuro. La generación de código se produce al llamar la función.*

Plantillas de función

- *Cuando se genera esta función, decimos que se está "instanciando" la plantilla, algo similar a la relación entre un objeto y su clase.*
- *Una gran ventaja de las plantillas es que si necesito corregir la función lo hago solo una vez.*
- *La memoria necesaria si se escriben plantillas o todas las funciones es la misma.*

Plantillas de función

- *La declaración de plantillas es un ejemplo claro de la filosofía POO (Programación Orientada a Objetos).*
- *Una plantilla es solo un modelo que servirá para generar las funciones reales.*
- *De la misma manera que una clase es un modelo que sirve para generar instancias que son los objetos.*

Plantillas con múltiples argumentos

- *Las plantillas de funciones pueden tener argumentos múltiples, tal como se verá en el ejemplo siguiente.*
- *Este contiene dos argumentos plantilla y uno de tipo básico.*
- *El compilador generará cuatro versiones de la función, que corresponde a las cuatro llamadas con argumentos de tipo diferente.*

```
#include <iostream>  // P05-05.cpp
using namespace std;
template <class atype>
int find( atype* array, atype value, int size ) {
  for(int j=0; j<size; j++)
    if(array[j]==value)
      return j;
    return -1;
}
```

```
Salida:
 5 en Arreglo char:   índice = 2
 6 en Arreglo int:    índice = -1
11 en Arreglo long:   índice = 4
 4 en Arreglo double: índice = -1
```

```
char chrArr[] =   {1, 3, 5, 9, 11, 13}; // arreglo
char ch = 5;                            // valor a encontrar
int intArr[] =    {1, 3, 5, 9, 11, 13};
int in = 6;
long lonArr[] =   {1L, 3L, 5L, 9L, 11L, 13L};
long lo = 11L;
double dubArr[] = {1.0, 3.0, 5.0, 9.0, 11.0, 13.0};
double db = 4.0;

int main( void ) {
  cout << " 5 en Arreglo char:   índice = " << find(chrArr, ch, 6);
  cout << "\n 6 en Arreglo int:    índice = " << find(intArr, in, 6);
  cout << "\n11 en Arreglo long:   índice = " << find(lonArr, lo, 6);
  cout << "\n 4 en Arreglo double: índice = " << find(dubArr, db, 6);
}
```

Plantillas con múltiples argumentos

- *Cuando se trabaja con múltiples argumentos debe tenerse en cuenta lo siguiente:*

```
int find( atype* array, atype value, int size )
```

- *El primero y el segundo argumento deben coincidir en tipo, por lo que:*

```
int intarray[] = { 1, 3, 5, 7 };
float f1 = 5.0;
int value = find( intarray, f1, 4 );
```

- *no es válido, porque la función espera:*

```
find( *int, int, int );
```

y no

```
find( *int, float, int );
```

Caso: más de un argumento plantilla

- *No estamos limitados a que la función tenga un solo argumento plantilla.*

```
template <class atype, class btype>
btype find( atype* array, atype value, btype size ) {
  for( btype j=0; j<size; j++ )   // observar el uso de btype
    if( array[ j ] == value )
      return j;
    return (btype )-1;
}
```

- *En este caso el contador puede ser de un tipo estándar o definido por usuario.*

Cuando usar plantillas

- *¿Cómo saber cuando puedo usar una plantilla?*
- *La pregunta se refiere a si funcionará bien en todos los casos que quiero utilizarla.*
- *Primera regla: no usar operadores que normalmente no trabajarían con el tipo de datos a enviar a la función.*
- *Por ejemplo, no usar == para comparar strings, ya se que eso no se usa.*

Cuando usar plantillas

- *Lo anterior no es insalvable, podría hacerlo si primero sobrecargo el operador == de manera que compare strings.*
- *Como se puede ver esto no se hace en forma automática.*
- *Es el programador que ha verificado primero que debe realizar esa modificación para que el operador sea compatible.*

Funciones Amigas

Funciones Amigas

- *Según algunos autores la idea de funciones amigas rompe los conceptos de encapsulación y abstracción aprendidos hasta ahora.*
- *Los más indulgentes dicen que las funciones amigas son una concesión a la realidad.*
- *En pocas palabras algunas excepciones a las reglas hacen la vida más fácil.*

Funciones Amigas

- *Las funciones amigas tienen acceso a los miembros privados de una clase, sin ser un método de la clase.*
- *Una función aislada puede se declarada como amiga, así como miembro de otras clases.*
- *No pueden ser funciones amigas ni los constructores ni los destructores.*

```
#include <iostream.h>  // P05-06.cpp
using namespace std;
class airtime
{
   int hours;              // 0 a 23
   int minutes;            // 0 a 59
  public:                    // constructor para 0, 1 o 2 arg
    airtime(int h = 0, int m = 0) : hours(h), minutes(m) {  }
    void display()          // salida a pantalla
      { cout << hours << ':' << minutes; }
    void get() {            // entrada del usuario
      char dummy;
      cin >> hours >> dummy >> minutes;
    }                     // operator + sobrecargado
    airtime operator + (airtime right) {
      airtime temp;         // objeto temporario
      temp.hours = hours + right.hours; // agregado de datos
      temp.minutes = minutes + right.minutes;
      if(temp.minutes >= 60) {     // verificación de acarreo
        temp.hours++;
        temp.minutes -= 60;
      }
      return temp;          // retorna objeto temporario por valor
    }
};
```

```
int main( void )
{
  airtime at1, at2;

  cout << "Ingrese una hora [10:00]: ";
  at1.get();

  at2 = at1 + 3;    // suma entero al objeto

  cout << "airtime + 3 = ";
  at2.display();

  // at2 = 3 + at1;   // error: illegal structure operation

}
```

```
Salida:

Ingrese una hora [10:00]: 11:45
airtime + 3 = 14:45
```

Funciones Amigas

- *Observar con mucha atención que el 3 en el programa anterior no es un entero que se suma a un objeto, se hace un objeto a partir del entero (aprovechando el constructor con un argumento) y después se suma como objeto al primero.*
- *Esto se puede verificar haciendo la siguiente modificación al constructor del programa anterior*

```
#include <iostream>
using namespace std;
class airtime
{
   int hours;              // 0 a 23
   int minutes;            // 0 a 59
  public:                    // constructor para 0, 1 o 2 arg
    airtime(int h = 0, int m = 0) : hours(h), minutes(m)
      { cout << "Constructor: " << hours << ":" << minutes << endl; }
    void display()          // salida a pantalla
      { cout << hours << ':' << minutes; }
    void get() {             // entrada del usuario
      char dummy;
      cin >> hours >> dummy >> minutes;
    }                      // operator + sobrecargado
    airtime operator + (airtime right) {
      airtime temp;         // objeto temporario
      temp.hours = hours + right.hours; // agregado de datos
      temp.minutes = minutes + right.minutes;
      if(temp.minutes >= 60) {     // verificación de acarreo
        temp.hours++;
        temp.minutes -= 60;
      }
      return temp;          // retorna objeto temporario por valor
    }
};
```

Funciones Amigas

- *En la función main() teníamos el comentario siguiente:*

```
// at2 = 3 + at1;   // error: illegal structure operation
```

  ¿Qué significa?

- *Significa que en este caso no convierte el 3 a objeto para hacer la "suma" al objeto at1. ¿porqué?*
- *Porque el operador sobrecargado es un método del objeto de la izquierda.*

Funciones Amigas

- *Este método o función miembro hace la "suma sobrecargada" del objeto que tie-ne a la derecha y en el ejemplo anterior interpretó que el 3 era la definición de un objeto con un solo argumento.*
- *Si lo que está a la izquierda es el 3, es obvio que no pasará lo mismo.*
- *Para solucionar este problema usamos una función amiga.*

```
#include <iostream>  // P05-07.cpp
using namespace std;
class airtime
{
   int hours;              // 0 a 23
   int minutes;            // 0 a 59
  public:                    // constructor para 0, 1 o 2 arg
    airtime(int h = 0, int m = 0) : hours(h), minutes(m) {  }
    void display()          // salida a pantalla
      { cout << hours << ':' << minutes; }
    void get() {            // entrada del usuario
      char dummy;
      cin >> hours >> dummy >> minutes;
    }                     // operator + sobrecargado
    friend airtime operator+(airtime, airtime); // declaración
};

airtime operator + (airtime left, airtime right) {
  airtime temp;      // objeto temporario
  temp.hours = left.hours + right.hours; // agregado de datos
  temp.minutes = left.minutes + right.minutes;
  if(temp.minutes >= 60) {         // verificación de acarreo
    temp.hours++;
    temp.minutes -= 60;
  }
  return temp;       // retorna el objeto temporario por valor
}
```

```
int main( void )
{
  airtime at1, at2;

  cout << "Ingrese una hora [10:00]: ";
  at1.get();

  at2 = at1 + 3;    // suma entero al objeto
  cout << "airtime + 3 = "; at2.display();
  cout << endl;

  at2 = 3 + at1;
  cout << "3 + airtime = "; at2.display();
  cout << endl;
}
```

```
Salida:

Ingrese una hora [10:00]: 11:45
airtime + 3 = 14:45
3 + airtime = 14:45
```

Funciones Amigas

- *Observar que dentro de la declaración de clase se agregó una función amiga que resuelve el operador sobrecargado.*
- *Esta función es declarada fuera de la clase y no está relacionada a ella, excepto que la clase la reconoce como función amiga.*

Funciones Amigas

- *La declaración de función amiga puede ir en cualquier lugar de la clase.*
- *Podría haber sido incluso en la parte privada.*
- *Conceptualmente debería en la parte pública porque corresponde a la inter-faz de la clase.*
- *Observar que la función amiga toma dos argumentos y no uno.*

Funciones Amigas

- *El uso de funciones amigas no es lo más deseable, excepto en situaciones como la analizada.*
- *Una clase tiene la protección de que si no declara a una función como amiga, nadie puede asociarle un "amigo indeseado".*
- *Si estamos usando en nuestro diseño muchas funciones amigas es señal que debemos revisar nuestro planteo.*

Funciones Amigas

- *Otra situación donde se utiliza funciones amigas es cuando queremos tener una notación más clara que la obtenida por objetos.*
- *Es posible que por ejemplo para calcular un cuadrado especial, se quiera utilizar una función que sea convocada de la misma forma que se convoca sqr().*
- *Veamos el ejemplo siguiente:*

```
#include <iostream>  // P05-08.cpp
using namespace std;
class medingl {
    int pies;
    float pulg;
  public:
    medingl(int ft, float in)  { pies = ft; pulg = in; }
    void showdist() { cout << pies << "\' " << pulg << '\"'; }
    float cuadr();
};

float medingl::cuadr() {
  float fltfeet = pies + pulg / 12;
  float feetsqrd = fltfeet * fltfeet;
  return feetsqrd;
}

int main( void ) {
  medingl dist(3, 6.0);
  float sqft = dist.cuadr();  // Esto a veces no es lo que se quiere
  cout << "Distancia = "; dist.showdist();
  cout << "\nCuadrado = " << sqft << " pies cuadrados\n";
}
```

```
Salida:
Distancia = 3' 6"
Cuadrado = 12.25 pies cuadrados
```

```
#include <iostream>  // P05-09.cpp
using namespace std;
class medingl {
    int pies;
    float pulg;
  public:
    medingl(int ft, float in)  { pies = ft; pulg = in; }
    void showdist() { cout << pies << "\' " << pulg << '\"'; }
    friend float square( medingl );    // función amiga
};

float square ( medingl mi ) {
  float fltfeet = mi.pies + mi.pulg / 12;
  float feetsqrd = fltfeet * fltfeet;
  return feetsqrd;
}

int main( void ) {
  medingl dist(3, 6.0);
  float sqft = square( dist );     // Aparece como una función común
  cout << "Distancia = "; dist.showdist();
  cout << "\nCuadrado = " << sqft << " pies cuadrados\n";
}
```

La salida es la misma que el caso anterior.

Funciones Amigas

- *El último caso que veremos es cuando la función necesita utilizar datos que corresponden a dos clases distintas.*
- *En esta circunstancia la función resultará amiga de ambas clases.*
- *En el ejemplo siguiente se puede observar la predeclaración de la clase beta, necesaria para poder declarar la función amiga frifunc() en la clase alfa*

```
#include <iostream>  // P05-10.cpp
using namespace std;
class beta;    // necesaria para la declaración de frifunc
class alfa {
    int data;
  public:
    alfa()  { data = 3; }  // constructor
    friend int frifunc( alfa, beta );
};

class beta {
  int data;
  public:
    beta()  { data = 7; }   // constructor
    friend int frifunc( alfa, beta );
};

int frifunc( alfa a, beta b ) {
  return( a.data + b.data );
}

int main( void ) {
  alfa aa;
  beta bb;
  cout << frifunc( aa, bb ) << endl;
}
```

```
Salida:
10
```

Clases Amigas

Clases Amigas

- *De la misma manera que tenemos funciones amigas, también podemos tener clases amigas.*
- *El principal motivo para tener clases amigas es la comunicación interclases.*
- *El escenario es cuando una clase está intimamante ligada a otra y necesita acceder a sus datos privados, la única manera de hacerlo es que sean amigas.*

Clases Amigas

- *Descartamos que no queremos por este motivo que una clase haga públicos sus datos privados, solo serán accesibles a la clase amiga.*
- *Tampoco podemos utilizar la herencia porque si bien las clases tienen estrecha relación, ninguna deriva de la otra, ni es razonable forzar las cosas para que sea de esta manera.*

Clases Amigas

- *Un problema a resolver en ICC (Inter Class Communication) es cómo se declaran las clases.*
- *En primer lugar vamos a ver como accede una clase amiga a los datos privados de la que brinda amistad.*
- *Debe recordarse que para acceder a datos de una clase que no es propia ni derivada se necesita tener objetos.*

```
#include <iostream.h>

class alfa {
  friend class beta;          // así beta puede acceder alfa
  int adata;                  // esta es la zona privada de alfa
};

class beta {
  public:
  void bfunc() {
    alfa objA;                // debe hacer un objeto para
    objA.adata = 3;           // acceder datos privados de alfa
  }
};
```

- *La declaración:* `friend class beta;` *está en la parte privada de alfa. Si bien esto no es necesario que sea así, conceptualmente es correcto considerar esta amistad como privada. No tiene relación con la interfaz.*

Clases Amigas

- *¿Qué sucede si en el caso anterior miembros de alfa necesitan acceder a datos privados de beta?*
- *Se debe declarar primero ambas clases y por último definir los métodos de alfa. O al menos el que accede a miembros privados de beta.*
- *Esto se muestra en el ejemplo siguiente:*

```
class alpha {
  public:
    void afunc();          // declaración de función
};                         // la definición debe estar después
                           // de la declaración de beta
class beta {
  friend class alpha;
  int bdata;
};

void alpha::afunc() {
  beta objB;
  objB.bdata = 3;
};
```

- *Observar que la definición del método afunc(), que tiene acceso a los miembros privados de beta, tiene lugar después de la declaración de beta.*

Amistad mutua con punteros

- *El uso de punteros en la relación inter-clases es muy común.*
- *A decir verdad es la forma más utilizada.*
- *En el ejemplo que sigue, continuando con las clases alfa y beta, vamos a suponer que cada objeto alfa está asociado con dos objetos beta.*
- *Esto significa que cada objeto alfa tiene dos punteros a objetos beta.*

Amistad mutua con punteros

- *Por otra parte cada objeto beta está relacionado con un objeto alfa.*
- *Esto significa que cada objeto beta tiene un puntero a objeto alfa.*
- *Por supuesto que en la declaración de ambas clases debemos declarar la amistad con la otra.*
- *Debe tenerse cuidado en el orden de definición de las funciones.*

```
#include <iostream>  // P05-11.cpp

using namespace std;

class beta;

class alpha {
    friend class beta;
    beta* bptr1;           // punteros a beta
    beta* bptr2;
    int adata;
  public:
    alpha();               // constructor (definido después de beta)
    void afunc();          // función (definida después de beta)
    void showa();
};

class beta {
    friend class alpha;
    alpha* aptr;
    int bdata;
    // observar que el constructor es privado
    beta(alpha* ap) : aptr(ap) { cout<< "Constructor de beta"<< endl; }
  public:                  // inicializa el puntero a alfa
    void bfunc() {
      aptr->adata = 3;     // accede datos privados de alfa
    }
};
```

```
alpha::alpha() {              // constructor de alfa
  bptr1 = new beta(this);
  cout << "Constructor de alpha, generé puntero bptr1 a beta" <<  endl;
  bptr2 = new beta(this);
  cout << "Constructor de alpha, generé puntero bptr2 a beta" << endl;
}

void alpha::afunc() {         // función de alfa
  bptr1->bdata = 4;           // accede datos privados de beta
  bptr2->bdata = 5;           // debe ser definida después de beta
}

void alpha::showa() {
  cout << endl << "Muestra de datos de alpha" << endl;
  cout << "btpr1->bdata: " << bptr1->bdata << endl;
  cout << "btpr2->bdata: " << bptr2->bdata << endl;
}

int main( void ) {
  alpha objA;
  objA.afunc();
  objA.showa();
}
```

```
Constructor de beta
Constructor de alpha, generé puntero bptr1 a beta
Constructor de beta
Constructor de alpha, generé puntero bptr2 a beta

Muestra de datos de alpha
btpr1->bdata: 4
btpr2->bdata: 5
```

Amistad mutua con punteros

- *La regla general es que la definición de funciones (aún de los constructores) debe estar después de las declaraciones de ambas clases.*